Campaign Events: Engineering assignment

Time box: 3.5–4 hours. This assignment deliberately contains more than most people finish. We do not expect everything to be done. Prioritise what you believe matters most, and tell us what you skipped and why. A smaller solution you fully understand beats a larger one you don't.

AI tools: You may use ChatGPT, Claude, Cursor, Copilot, documentation, Google, or anything else you normally use. We assume you will. What we evaluate is whether *you* understand the problem and the code you submit. See the AI usage section at the end.

Stack: Go for the backend. SQLite or in-memory persistence — your choice. A frontend is entirely optional and earns no extra credit; do not spend time on UI polish.

The business, in sixty seconds

Relay (fictional) sends marketing messages — email, SMS, push — on behalf of its clients. Relay itself doesn't deliver anything; it hands messages to third-party delivery providers. As things happen to each message, the provider calls Relay's webhook with an **event**.

There are four event types: sent, delivered, opened, clicked.

Marketers watch a live dashboard while their campaign runs. The dashboard needs numbers per campaign. You own the service that receives events from providers and answers the dashboard's questions.

Two facts of life in this business:

1. **Providers retry.** If they don't get a timely 2xx from us, they send the same event again. Sometimes they send it again anyway.
2. **Events arrive late and out of order.** An opened can arrive before the delivered for the same message. Events can show up hours after they happened.

The brief (from your product manager)

> We hand messages to delivery providers, and they call our webhook with events as things happen. I need an API the dashboard can call to show campaign stats — how many were sent, delivered, opened, clicked. Providers retry, so the same event can arrive more than once, and the numbers must not double count. Traffic today is

> a few thousand events a day across ~50 active campaigns, but it's growing. Can you build the service that receives events and serves the numbers?

That's the whole brief. It is intentionally not a complete specification.

The event payload

This part *is* fixed — it's the contract with providers:

```
{
  "event_id": "evt_00042",
  "campaign_id": "cmp_summer_sale",
  "contact_id": "ct_017",
  "type": "opened",
  "timestamp": "2026-08-10T09:15:04Z",
  "metadata": {"url": "https://example.com/offer"}
}
```

event_id is assigned by the provider and identifies one event. timestamp is when the event happened at the provider, not when it reached us. metadata is optional.

Part 1: Read and think first (15–20 minutes)

Before writing any code, create NOTES.md and write down:

1. Your interpretation of the problem in your own words.
2. The assumptions you are making.
3. Ambiguities you noticed in the brief.
4. The questions you would ask the PM if you could.
5. What you will prioritise in the time available, and in what order.

Keep it plain and honest. Bullet points are fine. We read this section first, and it carries real weight in evaluation. There is no trick: the brief genuinely leaves things undecided, and we want to see whether you notice.

Part 2: Build it (~2 hours)

Implement the service. Required:

- POST /events — accepts a JSON array of events from providers.

- GET /campaigns/{campaign_id}/stats — returns statistics for one campaign. You design the response shape; ask yourself what a dashboard actually needs.

Optional, only if you have time:

- GET /campaigns/{campaign_id}/events — a paginated list of events, e.g. for a "recent activity" view.

A starter skeleton is in starter/ (compiling Go HTTP server with stub handlers, stdlib only). Use it, restructure it, or discard it.

starter/seed/events.json contains a day of provider traffic — including the mess real providers send. Your running service should survive everything in that file. Surviving does not mean accepting: what to reject, and how, is one of your decisions.

Decisions we will pay attention to (we are not telling you the right answers — several are defensible if you argue them):

- What exactly makes two events "the same"?
- What should the API return when it receives a duplicate? An invalid event? A batch containing both good and bad events?
- What does "how many opened" mean, precisely?
- How do you store events, and why that way?
- What happens when two requests hit your service at the same time?

Write tests where they earn their keep — the behaviour you'd be most embarrassed to break. Do not chase coverage.

One practical note: if you decode the whole request body straight into []Event, think about what happens to the other 199 events when one of them is malformed.

Part 3: Debug someone else's code (30–40 minutes)

The debugging/ folder contains a small, self-contained Go program that computes campaign statistics from a file, its input (events.jsonl), and expected_output.txt — the exact output a correct version produces.

The program compiles and runs. It is also wrong. There are **four bugs**. None of them is a syntax error, and at least one only shows itself under concurrency — go run -race . events.jsonl is your friend.

The rules and the expected behaviour are in debugging/README.md. For each bug, write in BUGS.md:

1. What the bug is.
2. Why it happens.
3. Your fix (make minimal changes — do not rewrite the program).
4. How you verified the fix.

When all four are fixed, `go run . events.jsonl` should match `expected_output.txt` exactly, every run.

Part 4: Scale memo (~15 minutes, written)

No code. Add a section to NOTES.md.

Your service handles a few thousand events a day. A large client signs, and the projection becomes **100 million events a day**. Answer briefly, in your own words:

- What breaks first in your current implementation, specifically?
- What would you change, and in what order?
- Would the API contract change? Would your storage design change?
- Would you introduce a queue? What new problems does that create?
- Where can duplicates now sneak in that couldn't before?
- How would you keep the counters trustworthy?
- What would you monitor?
- What would you deliberately *not* solve yet?

We are evaluating reasoning, not vocabulary. "I don't know, but here is how I'd find out" is a legitimate answer.

Part 5: The angry marketer (~10 minutes, written)

At 10:00 a marketer's dashboard shows:

Sent: 1,000,000 Delivered: 970,000 Opened: 250,000 Clicked: 20,000

At 10:30 it shows:

Sent: 1,000,000 Delivered: 975,000 Opened: 248,000 Clicked: 20,000

Delivered went **up**. Opened went **down**. The marketer files a ticket: "Your dashboard is broken."

In NOTES.md: list every plausible explanation you can think of, before assuming the code is broken. Then say what you would check first, and how you would decide whether this is a bug or expected behaviour. Also: is the delivered number rising actually suspicious?

AI usage disclosure

Include AI_USAGE.md with:

- Which tools you used.
- Roughly what you used them for.
- One suggestion from AI that you rejected or changed, and why.
- One thing AI helped you understand.
- Anything AI generated that you then had to debug.

Heavy AI usage costs you nothing. Submitting code you cannot explain costs you everything — the follow-up interview walks through your code, live.

Submission

A git repo or zip containing:

```
NOTES.md          Parts 1, 4, 5 + anything else you want us to know

your service      source, and a README with exact run instructions

BUGS.md           Part 3 findings

debugging/        with your fixes applied

AI_USAGE.md
```

End NOTES.md with two short sections: **"What I completed / what I intentionally skipped"** and **"If I had another day"**.

Do not set up deployment, Kubernetes, cloud accounts, or authentication. Docker is optional and unnecessary. If go run . works and your README says how to hit the endpoints, you're done.

Suggested time budget

Part	Time
1. Read and think	15–20 min
2. Build	~2 h
3. Debug	30–40 min
4. Scale memo	~15 min
5. Angry marketer	~10 min

If you're running out of time, cut scope in Part 2 and say so in NOTES.md. Do not skip Parts 1, 4, or 5, they're short, and they're where we learn how you think.